

Complex Simulation

多规则、多状态模拟

状态维护 · 规则分派 · 队列与标记 · P5661 公交换乘精讲

把题面里互相缠绕的规则一条条拆开，把随时变化的状态一个个管住——规则不再只有一条，这就是模拟的第二课。

02

02

授课班级

中预 2026 级 B 班

主讲

信息教研组

时间

2026 年 10 月

共 10 道课堂实践题 · 含洛谷题号

学习目标与课堂安排

1 认识"状态"

找出程序要随时维护的信息：手里的票、每个龙头剩几秒、当前朝向……先列状态清单，再动手写代码。

2 会做规则分派

题面有多条规则时，按事件发生顺序写主循环，每条规则一个 if 分支，先后优先级以题面为准。

3 会管"会过期的东西"

用队列维护先后顺序，用标记数组记"已用 / 作废"，过期对象从队头离开——票夹、传令、淘汰都靠它。

4 有复杂度意识

动手前先估算：循环要跑多少次？数据到 10 万时，盲目套循环会超时，"每步只做一点事"才稳。

课堂安排

建议节奏，可按学情调整

- 15 分钟 概念讲解
- 25 分钟 例题精讲 P5661
- 55 分钟 上机实践（10 题）
- 10 分钟 总结答疑

承上启下

第 1 课我们学会了"照题意一步步做"；这一课的模拟会同时转起好几条规则、好几个状态。

什么是多规则、多状态模拟？

一句话定义

当题面同时出现"好几条规则"和"好几份要随时维护的信息"，模拟就从"照做"升级为"管好状态、分派规则"。

1 状态要找全

把"随时会变的信息"列成清单，一个都不能少。P5661 里每张优惠票都有三个属性：面值、获得时刻、是否已用。

2 规则要分派

每条规则翻译成一个分支；分支之间有优先级——先用票还是先花钱？谁先谁后，题面说了算，不能自己想当然。

3 顺序是主轴

主循环按"事件发生的时间顺序"推进：一次事件发生了，就依次执行它触发的所有规则，一步都不打乱。

生活里的一次"多规则模拟" —— 地铁换乘优惠（今天核心题的原型！）



C++ 视角

读入数据 → 维护状态 → 按规则分支 → 输出结果。
CSP-J 入门级 T2、NOIP 普及组 T1/T2 大多是这类题，把细节理清就是最稳的得分手段。

关键点一：先把"状态"找出来

什么是"状态"?

程序运行到任意时刻, "接下来怎么算"只取决于的那几个量, 就是**状态**。
状态一旦找对, 剩下的就是一步步推演。

接水问题里, 谁在哪个龙头、还要接几秒——决定下一秒发生什么。所以我们选: **finish[j] = j 号龙头上的人"接完的时刻"**。

状态清单怎么列?

问自己: "如果程序此刻停住, 要记住哪些数才能继续算?" 每个状态对应一个变量 / 数组 / 队列, 写进注释里。

方法总结

想清楚"程序要随时记住什么", 代码就完成了一半。下一页的关键点二、三, 都在为"状态怎么维护"服务。

样例执行过程 (n=5 人, m=3 个龙头, 接水量 4 4 1 2 1)

动作	各龙头接完时刻 finish	说明
1~3 号上龙头	4 4 1	前 m 人各占一个龙头
4 号 (要 2 秒)	4 4 3	找 finish 最小的龙头 3, 接上
5 号 (要 1 秒)	4 4 4	仍是龙头 3 最早空出
输出答案	max = 4	最晚空出的龙头决定总时间

注意"状态"选得好不好, 直接决定代码好不好写: 记"接完时刻", 新同学来了只要**找最小、加一笔**; 答案就是所有时刻的最大值。

关键点二：把规则写成有优先级的分支

乘公交那一刻，规则有先后——查票三支

- 1

先清队头：队头的票已过期（时间差超过 45）或已使用 → 它永远不会再派上用场，直接出队。
- 2

从早到晚找票：在剩下的票里找第一张"没过期、没用过、面值 $\geq$ 公交票价"的票——找到就**免费乘车**，并把这张票标记为已用。
- 3

找不到才花钱：没有可用票时自己购票。地铁记录永远是"花钱 + 获得新票"，根本不进这个查票流程。

为什么"先清队头"？

时间只增不减，队头的票最早获得、也最先过期；已用的票同样没有价值。把它们请走，队列里全是"还有机会"的票。

主循环怎么写？

按时间顺序读入 n 条乘车记录，一条记录 = 一次事件：

$op == 0$ （地铁）→ 花钱 $cost += p$ ，票夹入队一张新票

$op == 1$ （公交）→ 清队头 → 找票 → 用票或花钱

两条记录的先后次序就是代码的执行次序——**顺序即算法**。

常见错序

先找票、后清队头：过期票占着队头，白白多扫好几遍；更糟的是把"地铁"也拉来查票，整题全错。规则的顺序，题面早就写好了。

关键点三：队列与标记，管好"会过期"的东西

P5661 的票夹：一条"从早到晚"的队列



从早到晚 ↑

A 武器一 · 队列（先进先出）

票按获得时间排队，"从早到晚找票" = 从队头向后扫。队头最早获得、也最先过期——废票从队头出队即可，后面的票不受影响。

B 武器二 · 标记 used[]（惰性删除）

队伍中间的过期票 / 已用票没法立刻删——打上 used 标记，查票时跳过；等它轮到队头再真正出队。P7912 的 gone[]、P1518 的 vis 同理。

复杂度小账（P5661）

- ① 每一分钟至多 1 条乘车记录（时间保证递增）
- ② 票有效期 45 分钟 → 队里同时最多 46 张
- ③ 每次查票最多翻 46 张

$$O(46 \cdot n)$$

$n = 10^5 \rightarrow$ 约 5×10^6 步
1 秒内轻松跑完 ✓

动手前的第一步：算账

模拟题总代价 $\approx$ 每条记录的平均代价 $\times$ 记录数。估出来超过 10^8 ，就要换更聪明的维护方式——这正是"队列 + 标记"存在的意义。

本课十题总览

编号	题目	来源	难度	核心状态与规则
T01	公交换乘 P5661	CSP-J 2019 T2	★★★	票队列（面值 · 过期 · used）：地铁买票 / 公交查票
T02	接水问题 P1190	NOIP2010 普及组	★★	finish[m] 龙头结束时刻：谁先空出谁接班
T03	乒乓球 P1042	NOIP2003 普及组	★★	当前局比分：达分且领先 2，11/21 双赛制
T04	石头剪刀布 P1328	NOIP2014 普及组	★★	周期序列取模 + 5×5 胜负查表
T05	铺地毯 P1003	NOIP2011 提高组	★★	每张毯矩形范围：顺序扫描，能盖就刷新答案
T06	玩具谜题 P1563	NOIP2016 普及组	★★★	当前下标 + 朝向：2×2 合并成"同减异加"
T07	两只塔姆沃斯牛 P1518	USACO 经典	★★★	六元组（双位置+方向）：移动规则 + 状态判重
T08	神奇的幻方 P2615	NOIP2015 普及组	★★★	当前位置 (x,y)：四条构造规则按序特判
T09	直播获奖 P7072	CSP-J 2020 T2	★★★	分数桶 cnt[0..600]：从高往低累加找第 p 名
T10	小熊的果篮 P7912	CSP-J 2021 T4	★★★	段头队列 + 链表摘除：每轮各段取最左

全部 10 题的参考代码均已通过洛谷官方样例验证。课堂节奏：T01 精讲 5 页；T02 - T10 每题 4 页（题面 → 思路 → 方法 → 代码），可按学情取舍。

精讲 T01 | 公交换乘 (P5661)

第 2 课 · 多规则、多状态模拟

CSP-J 2019 T2 · 难度 ★★★

题面描述

某旅游城市收集到 n 条乘车记录，**按时间先后**给出。记录分两种：乘地铁 ($op=0$)、乘公交 ($op=1$)，每次出行的票价为 p 元、时刻为 t 。

乘地铁：必须自己买票——付出票价 p ，并获得一张**优惠票**：面值等于这次地铁票价 p ，有效期 45 分钟（从乘车时刻 t 起，到 $t+45$ 及之前的时刻有效）。

乘公交：若手中有**未使用、未过期、面值 $\geq$ 公交票价**的优惠票，则使用**获得最早**的那张免费乘车；否则自己花 p 元购票。每张优惠票最多使用一次。

地铁 = 花钱 + 领票（永远不查票）；公交 = 先查票（能免则免），没票才花钱。

思路预告

第 1 课的模拟只有两三个状态；这题里每张票都有自己的面值和到期时间——状态从“一盆”变成“一队”。先别写代码，把一张票的一生画出来。

输入格式

第一行一个整数 n 。接下来 n 行，每行三个整数 $op\ p\ t$ ： $op=0$ 地铁、 $op=1$ 公交。

输出格式

一行一个整数：完成全部出行的最少花费。

数据范围

$1 \leq n \leq 10^5$ ； $1 \leq p \leq 10^3$ ； $1 \leq t \leq 10^9$ ；记录按时间先后给出。

样例 1 (答案 36)

输入

```
6
0 10 3
1 5 46
0 12 50
1 3 96
0 5 110
1 6 135
```

输出

36

用票过程见第 10 页
手推表

题面 → 代码：主循环与查票三步

主循环伪代码：一条记录，做一件事

```
读入 n
for i = 1 .. n:           // 一条记录一件事
    读入 op, p, t
    if op == 0:           // ① 地铁
        cost += p        // 花钱
        入队新票(面值 p, 到期 t+45) // 领票
    else:                 // ② 公交
        while 队头已用或已过期: 出队 // 清队头
        找第一张 未用 · 未过期 · 面值够 的票
        if 找到: 标记 used    // 免费乘车
        else: cost += p      // 自己买
输出 cost
```

一句话

地铁入队、公交查票；时间递增让“清队头”永远正确——把规则翻译成分支，这就是全部。

三个细节追问

Q1 · 为什么用“获得最早”的那张？

题面白纸黑字。票按获得时间排队，“从早到晚找”恰好就是队列顺序——数据结构替我们记住了规则。

Q2 · 队中间的过期票谁来清？

清队头只处理最前面的。中间的票在扫描时逐张检查“未过期且面值够”，不合格直接跳过——不会出错。

Q3 · 用过的票为什么不立刻出队？

它在队中间，出队要整体搬移；标记 used 零成本，等它自己挪到队头再被清走。标记管“死活”，队列管“先后”。

状态设计 + 样例手推：一步步算出 36

状态清单：程序里要维护什么

- 1 票夹队列 `q`：存 (面值, 到期, used)，按获得时间从早到晚排列。
- 2 `head / tail`：队头只出（过期或已用）、队尾只进（地铁领票）。
- 3 `cost`：累计花费，统一用 `long long` 最稳（`t` 可达 10^9 ）。
- 4 每条记录 `op p t`：一次输入 = 一次状态转移。

手推 = 人肉当 CPU

左列写状态，右列写动作，一行一行走——写代码前先在纸上通一遍样例。

记录 (op p t)	动作	票夹 (面值 · 到期)	本次	累计
0 10 3	地铁：花 10，领票①	① 10 · 48	10	10
1 5 46	票①可免 ($46 \leq 48$)	① 已用	0	10
0 12 50	地铁：花 12，领票②	①②	12	22
1 3 96	①②均过期，买票	空 (①②清出)	3	25
0 5 110	地铁：花 5，领票③	③ 5 · 155	5	30
1 6 135	③面值 $5 < 6$ ，买票	③ 留在队中	6	36

- 6 条记录全部走完，累计花费 **36** = 官方样例 1 答案 (verify 脚本实测通过)。样例 2 答案是 32，课后按同样的表自己推一遍。

参考代码：手写队列 + used 标记

第 2 课 · 多规则、多状态模拟

课堂例 T01 · 公交换乘 (P5661)

```
#include <iostream>
using namespace std;
struct Ticket { int face; long long expiry; bool used; };
Ticket q[100005]; // 票夹：按获得时间从早到晚
int head = 0, tail = 0; // 手写队列的队头 / 队尾
int main() {
    int n; cin >> n;
    long long cost = 0; // 总花费，开 long long 稳
    for (int i = 0; i < n; i++) {
        int op; long long p, t;
        cin >> op >> p >> t;
        if (op == 0) { // ① 地铁：花钱 + 领票
            cost += p;
            q[tail++] = { (int)p, t + 45, false };
        } else { // ② 公交：查票三步
            while (head < tail && (q[head].used || t > q[head].expiry)) head++;
            int pick = -1;
            for (int j = head; j < tail; j++)
                if (!q[j].used && t <= q[j].expiry && q[j].face >= p) { pick = j; break; }
            if (pick != -1) q[pick].used = true; // 3a 免费
            else cost += p; // 3b 买票
        }
    }
    cout << cost << endl; // 输出最少花费
    return 0;
}
```

结构一览

struct Ticket 三字段：面值 / 到期 / used。q 数组当队列，head、tail 两个下标搞定进出——第 3 课学完队列原理会更明白。

三步对号入座

while 行 = 清队头；for 行 = 从早到晚找第一张可用票；if / else = 免费标记或花钱。与第 9 页伪代码一一对应。

两个好习惯

① 时间和花费统一 long long (t 可达 10^9)；② 读入即处理，不需要把 n 条记录存下来二次扫描。

提交提示

课后亲手敲一遍并提交洛谷 P5661。两个官方样例 (36 / 32) 及全部测试点均已通过对拍脚本验证。

易错门诊 + 复杂度小账

第 2 课 · 多规则、多状态模拟

课堂例 T01 · 公交换乘 (P5661)

错1 过期判断写反或写严

有效条件是 $t - t_0 \leq 45$ ，注意是 " $\leq$ "。写成 $<$ 会把恰好第 45 分钟的票白白扔掉；而且过期是“用到时才判断”，票不会到点自己消失。

错2 用过的票直接出队

每张票只能免一次费，用完要标 `used=true`。它可能在队中间，直接出队要整体搬移，还可能把后面没过期的票一起弄丢。

错3 数据范围看漏，复杂度写炸

t 可达 10^9 ，时间差、到期时刻统一 `long long`； n 最多 10^5 条记录，“每买一张票就把整个票夹重扫+搬移”的 $O(n^2)$ 写法会超时。

复杂度小账

$$O(46 \cdot n)$$

票的寿命最多 45 分钟，时间又严格递增，所以队列里任何时刻只有“最近 45 分钟”获得的票——清理加扫描，每条记录约 46 次操作。 $n = 10^5$ 时总操作约 5×10^6 ，时限内轻松通过。

本讲要点回收

多状态不可怕：每个状态配一个变量或数组，每条规则配一个分支，顺序读入顺序处理——T01 就是这套流程的完整示范。

T02 | 接水问题 (P1190)

第 2 课 · 多规则、多状态模拟

NOIP2010 普及组 · 难度 ★★

题面描述

水房里装了 m 个水龙头（编号 $1..m$ ），同一时刻每个龙头只能供一个人接水。 n 名同学按编号 $1..n$ 的固定顺序接水，第 i 名同学接水量为 w_i （耗时按 w_i 计）。

开局：前 m 名同学分别站到 $1..m$ 号龙头，立刻开始接水。

补位：后面的同学到达时若没有空龙头，就在龙头旁等待；哪个龙头先空出，等待者立刻上去。模拟时让下一位同学站到最早空出的龙头（并列取编号小的）。

求最后一名同学接完水的时刻。

思路预告

又是排队！但这次“服务窗口”有 m 个、每人耗时还不一样——状态从一个队列，变成了 m 个窗口各自的“结束时刻”。

输入格式

第一行两个整数 n, m 。第二行 n 个整数 $w_1 \ w_2 \ \cdots \ w_n$ ，按接水顺序给出。

输出格式

一个整数：最后一人接完水的时刻。

数据范围

$1 \leq n \leq 10^4$ ； $1 \leq m \leq 100$ （且 $m \leq n$ ）； $1 \leq w_i \leq 100$ 。

样例（两组，脚本已验证）

样例 1 输入 → 输出 4

5 3

4 4 1 2 1

样例 2 输入 → 输出 163

8 4

23 71 87 32 70 93 80 76

思路梳理：盯住每个窗口的"空出时刻"

第 2 课 · 多规则、多状态模拟

T02 · 接水问题 (P1190)

1

开局占位

前 m 名同学直接各占一个龙头： $\text{finish}[j] = w[j]$ ($j = 1..m$)。此时每个窗口的"结束时刻"就是第一个人的接水量。

2

逐个补位（核心动作）

对第 i ($i > m$) 位同学：扫 m 个龙头，用**打擂台**找 finish 最小的窗口 j （严格小于，并列自动取编号小的）；站上去： $\text{finish}[j] += w[i]$ 。

3

收尾报数

全部同学安排完，答案 = $\max(\text{finish}[1..m])$ ——最慢那个窗口接完的时刻，就是"最后一人接完水"的时刻。

一句话

窗口固定 m 个、人来人往——只要盯着每个窗口"什么时候空"这一件事，谁来了都往最早空的窗口里放。

方法拆解 + 样例手推：答案是 4

状态与账本

唯一状态： $finish[1..m]$ ——每个龙头"当前这批人接完"的时刻。等待者到来不改状态，只有"站上去"才改。

为什么不用队列？ 接水顺序就是输入顺序，第 i 位同学永远按编号来——用循环变量 i 就够了，省一个队列。

复杂度： n 人 $\times$ 每人扫 m 个窗口 $= O(n \cdot m) \leq 10^6$ 次比较，毫秒级。

手推 = 人肉当 CPU

右边用样例 1 走一遍：先占位、再补位、最后取 $\max$ 。

同学 i	接水量 w	站到哪	该龙头结束	此刻 $finish$
1	4	龙头 1（占位）	4	4 4 1
2	4	龙头 2（占位）	4	4 4 1
3	1	龙头 3（占位）	1	4 4 1
4	2	龙头 3（最早空出）	$1+2 = 3$	4 4 3
5	1	龙头 3（最早空出）	$3+1 = 4$	4 4 4

● 收尾： $\max(4, 4, 4) = 4$ ✓ 与样例一致。样例 2 同法手推得 **163**（对拍脚本验证一致）。

易错

答案是所有窗口结束时间的最大值，不是"最后一位同学的编号"也不是等待时间之和；打擂台要用严格小于，并列时自动保住编号小的窗口。

参考代码：一个数组 + 打擂台

第 2 课 · 多规则、多状态模拟

T02 · 接水问题 (P1190)

```
#include <iostream>
using namespace std;
int w[10005], finish[105];
int main() {
    int n, m; cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> w[i];
    for (int i = 1; i <= m; i++) finish[i] = w[i]; // 前 m 人先占位
    for (int i = m + 1; i <= n; i++) {
        int j = 1; // 找最早空出的龙头
        for (int k = 2; k <= m; k++)
            if (finish[k] < finish[j]) j = k; // 严格小于：并列取编号小
        finish[j] += w[i]; // 站上去接水
    }
    int ans = 0;
    for (int i = 1; i <= m; i++) ans = max(ans, finish[i]);
    cout << ans << endl; // 最后一人接完的时刻
    return 0;
}
```

模板复用

中间三层循环就是第 1 课学的“打擂台找最小值”——模拟题的一大半代码，都是学过的基础积木。

两个细节

① $\text{finish}[k] < \text{finish}[j]$ 用严格小于，并列时编号小的窗口自动胜出；② 答案是 $\max(\text{finish})$ ，单个窗口的值都不代表全局。

想得更远

若 n 大到 10^5 、 m 也大， $O(n \cdot m)$ 会慢——那时用小根堆把“找最早空出”降到 $O(\log m)$ 。这是第 12 课的内容，先埋颗种子。

提交提示

课后提交洛谷 P1190。两组官方样例（4 / 163）与全部测试点均通过对拍脚本验证。

T03 | 乒乓球（P1042）

第 2 课 · 多规则、多状态模拟

NOIP2003 普及组 · 难度 ★★

题面描述

华华打乒乓球赛：记录中 **W** 表示华华得一分，**L** 表示对手得一分，按时间顺序逐球给出；**E** 表示记录结束。

一局结束的条件：一方得分达到 **base** 分且领先对方至少 2 分。两个条件缺一不可。

题目要求把**同一场比赛**分别按 **11 分制**和 **21 分制**输出：每局比分一行（a:b），最后不足一局的当前比分也要输出；两种赛制的结果之间**恰好空一行**。

思路预告

同一场比赛、两套规则，规则间只差一个"base 分"——把"结算一局"写成带参数的函数，一份逻辑跑两遍。

输入格式

若干行，每行若干个大写字母（W / L / E），读到 E 为止。球数不大，逐球模拟即可。

输出格式

先 11 分制每局 a:b；空一行；再 21 分制每局 a:b。

数据范围

记录若干行、总球数有限（ $\ll 10^6$ ），E 一定出现。

官方样例（脚本已验证）

输入（20 个 W 一行 + WWLWE）	输出
WWWWWWWWWWWWWWWWWWWW	11:0
WWLWE	11:0
	1:1
	21:0
	2:1

思路梳理：一份结算逻辑，跑两遍

第 2 课 · 多规则、多状态模拟

T03 · 乒乓球 (P1042)

1

逐球读入，存成一条流水

用 `cin` 一次读一个字符：自动跳过换行和空格，读到 `E` 就停。把 `W / L` 依次存进字符串 `s`——这就是比赛的完整流水。

2

结算函数 `work(base)`：把"规则差"变成参数

从头到尾扫一遍 `s`，累计双方 `a`、`b`；每球后判断：**`over = (a ≥ base 且 a-b ≥ 2) 或 (b ≥ base 且 b-a ≥ 2)`**。成立就输出 `a:b` 并双双清零。

3

末局照输出

扫描结束后，把剩余比分 `a:b` 照样输出——哪怕 `0:0`。主程序依次：`work(11)` → 输出一个空行 → `work(21)`。

一句话

两种赛制只有 `base` 一个差别——把差别提炼成函数参数，规则再多也只写一遍代码。

方法拆解 + 样例手推：对上官方输出

三个考点

- ① **"且"别写成"或"**：达到 base 分 $\neq$ 局终，还要领先至少 2 分；10:10 之后继续打，直到拉开 2 分。
- ② **逐字符读入**：cin 读 char 自动跳过空白；判断完 W/L 别忘了 E 是停止符，不参与计分。
- ③ **输出格式**：每局 a:b 一行；两部分之间恰好一个空行；末局不足 base 也要输出当前比分。

状态一览

一场比赛只需 a、b 两个计数器——比 T01 少得多，难点全在"且"字和格式。

球序	过程	11 分制	21 分制
1 - 11	连续 11 个 W	11:0 (达标+领先2)	a=11, 未到 21
12 - 22	又是 11 个 W	11:0	a=22, 未到 21? → 到了!
23 - 24	L、W	1:1 (末局照输出)	a=23 达标 → 21:0; 剩 W、L
收尾	扫描结束	——	2:1 (末局照输出)

与官方样例逐行对照 (脚本验证一致)

11 分制：11:0、11:0、1:1 → 空一行 → 21 分制：21:0、2:1。注意第 21 球结束时 21 分制这局刚达标，第 22 球开始已是新的一局。

参考代码：work(base) 一函数两赛制

第 2 课 · 多规则、多状态模拟

T03 · 乒乓球 (P1042)

```
#include <iostream>
#include <string>
using namespace std;
string s;
void work(int base) {           // 按 base 分制结算并输出
    int a = 0, b = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        if (s[i] == 'W') a++; else b++;
        bool over = (a >= base && a - b >= 2) || (b >= base && b - a >= 2);
        if (over) { cout << a << ":" << b << "\n"; a = b = 0; }
    }
    cout << a << ":" << b << "\n";    // 末局照输出 (可能 0:0)
}
int main() {
    char c;
    while (cin >> c && c != 'E') s += c;    // 读到 E 为止
    work(11);
    cout << "\n";                        // 两种赛制之间空一行
    work(21);
    return 0;
}
```

一函数两赛制

work(base) 的参数就是两套规则唯一的差别。第 2 课的关键手法提前出场：**把规则差变成参数**。

读入与停止

while (cin >> c && c != 'E')：一个字符一个字符读，E 既是数据结尾也是循环出口——多条件 while 的经典写法。

格式分也是分

冒号、每局一行、恰好一个空行、末局照输出——NOIP 早期题格式扣分狠，逐字符对样例。

提交提示

课后提交洛谷 P1042。官方样例（含空行）已通过对拍脚本逐字符验证。

T04 | 生活大爆炸版石头剪刀布（P1328）

题面描述

升级版猜拳：**0 剪刀、1 石头、2 布、3 蜥蜴人、4 斯波克**五种手势，任意两种手势之间的胜负关系题目全部给定（共 10 条）。

A、B 两人共玩 **N 轮**：A 按自己的出招周期 $A_1 \cdots A_{(NA)}$ **循环出招**，B 按周期 $B_1 \cdots B_{(NB)}$ 循环出招（NA、NB 可以不同）。

每轮查表计分：赢者得 1 分，平局无人得分。求 N 轮后两人的总得分。

思路预告

循环出招 = 下标取模；五种手势的两两关系 = 一张 5×5 表——把"查表"变成模拟的武器，规则就不用写十个 if。

输入格式

第一行 N NA NB；第二行 NA 个整数（A 周期）；第三行 NB 个整数（B 周期）。

输出格式

一行两个整数：A 的得分与 B 的得分。

数据范围

N、NA、NB 均不超过 200；出招为 0..4 的整数。

官方样例（脚本已验证）

样例 1 → 6 2

1	0	5	6
0	1	2	3
4	0	3	4
2	1	0	

样例 2 → 4 4

9	5	5
0	1	2
3	4	
1	0	3

考点一：把两两胜负写成 5×5 表

查表方向：行 = A 的手势 x，列 = B 的手势 y；蓝格 A 胜，红格 B 胜，灰格平局

A \ B	0 剪刀	1 石头	2 布	3 蜥蜴人	4 斯波克
0 剪刀	平	B胜 · 砸	A胜 · 剪	A胜 · 剪	B胜 · 驳
1 石头	A胜 · 砸	平	B胜 · 包	A胜 · 砸	B胜 · 驳
2 布	B胜 · 剪	A胜 · 包	平	B胜 · 食	A胜 · 包
3 蜥蜴人	B胜 · 剪	B胜 · 砸	A胜 · 食	平	A胜 · 毒
4 斯波克	A胜 · 气化	A胜 · 气化	B胜 · 驳	B胜 · 毒	平

怎么记住这张表？

每格就是一条规则：剪刀剪布/蜥蜴人，石头砸剪刀/蜥蜴人，布包石头/斯波克，蜥蜴人吃布/毒斯波克，斯波克气化剪刀/石头。代码里就是 `int wtab[5][5]`，一查 `O(1)`——表格就是压缩过的规则库。

考点二：周期取模 + 十轮手推（样例 1 → 6 2）

取模：让两个不同长度的周期对齐

第 i 轮 (i 从 0 起) : $x = a[i \% NA]$, $y = b[i \% NB]$ 。周期不同没关系，取模自动回卷。

判分三连: if (wtab[x][y]) sa++; else if (wtab[y][x]) sb++; else 平局，什么也不做。

手推两步走: 先写出 $i \% NA$ 、 $i \% NB$ 是多少，再查表定胜负——两步分开做，不会乱。

样例 1 的节奏

NA=5、NB=6: 第 6 轮回到 A₁，第 7 轮回到 B₁ ——右边从第 6 轮起就能看到"重来"。

轮	A 出招	B 出招	结果	累计 (A:B)
1	0 剪刀	0 剪刀	平	0:0
2	1 石头	3 蜥蜴人	A胜 · 砸	1:0
3	2 布	4 斯波克	A胜 · 包	2:0
4	3 蜥蜴人	2 布	A胜 · 食	3:0
5	4 斯波克	1 石头	A胜 · 气化	4:0
6	0 剪刀	0 剪刀	平	4:0
7	1 石头	0 剪刀	A胜 · 砸	5:0
8	2 布	3 蜥蜴人	B胜 · 食	5:1
9	3 蜥蜴人	4 斯波克	A胜 · 毒	6:1
10	4 斯波克	2 布	B胜 · 包	6:2

- A 胜第 2、3、4、5、7、9 轮共 6 分，B 胜第 8、10 轮共 2 分 → 输出 6 2 √（样例 2 同法得 4 4，脚本验证一致）。

参考代码：查表 + 取模，二十行搞定

第 2 课 · 多规则、多状态模拟

T04 · 生活大爆炸 (P1328)

```
#include <iostream>
using namespace std;
int wtab[5][5] = {           // wtab[x][y]==1 表示 x 胜 y
    {0,0,1,1,0},           // 剪刀: 胜 布、蜥蜴人
    {1,0,0,1,0},           // 石头: 胜 剪刀、蜥蜴人
    {0,1,0,0,1},           // 布: 胜 石头、斯波克
    {0,0,1,0,1},           // 蜥蜴人: 胜 布、斯波克
    {1,1,0,0,0}            // 斯波克: 胜 剪刀、石头
};
int a[205], b[205];
int main() {
    int n, na, nb; cin >> n >> na >> nb;
    for (int i = 0; i < na; i++) cin >> a[i];
    for (int i = 0; i < nb; i++) cin >> b[i];
    int sa = 0, sb = 0;
    for (int i = 0; i < n; i++) {
        int x = a[i % na], y = b[i % nb];    // 周期取模
        if (wtab[x][y]) sa++;
        else if (wtab[y][x]) sb++;
    }
    cout << sa << " " << sb << endl;
    return 0;
}
```

表是全局常量

wtab 在 main 外初始化，一行注释一个手势——抄表时横竖对照第 22 页，抄错一格全盘错。

判分的对称性

wtab[x][y] 没中，再查反方向 wtab[y][x]；两个都为 0 就是平局——不需要单独写平局分支。

从 0 开始更顺

数组下标与手势编号都从 0 起，第 i 轮就是 a[i % na]——少一串 ± 1 换算，是这张表最舒服的打开方式。

提交提示

课后提交洛谷 P1328。两组官方样例（6 2 / 4 4）与全部测试点均通过对拍脚本验证。

T05 | 铺地毯 (P1003)

第 2 课 · 多规则、多状态模拟

NOIP2011 提高组 · 难度 ★★

题面描述

房间里铺 n 张矩形地毯。第 i 张由左下角 (a_i, b_i) 、 x 方向长度 g_i 、 y 方向长度 k_i 描述，覆盖 $a_i \leq x \leq a_i + g_i$ 且 $b_i \leq y \leq b_i + k_i$ （含四条边）。

地毯按编号顺序铺设，后铺的盖在先铺的上面。给定地面一点 (x, y) ，求这一点最上面是第几号地毯；若没有任何地毯覆盖，输出 -1 。

思路预告

不用真的“叠”地毯——地毯之间只有一条规则：后铺的赢。一遍扫描、记住最后一个覆盖者就够了。

输入格式

第一行 n ；接下来 n 行每行 $a\ b\ g\ k$ 四个整数；最后一行是询问点 $x\ y$ 。

输出格式

一个整数：最上面地毯的编号，无覆盖输出 -1 。

数据范围

$0 \leq n \leq 10^4$ ； $0 \leq a, b, g, k \leq 10^5$ 。

官方样例（同一批地毯问两次）

询问 (2,2) $\rightarrow$ 3	询问 (4,5) $\rightarrow$ -1
3	3
1 0 2 3	1 0 2 3
0 2 3 3	0 2 3 3
2 1 3 3	2 1 3 3
2 2	4 5

思路梳理："后铺的赢"怎么变成代码

第 2 课 · 多规则、多状态模拟

T05 · 铺地毯 (P1003)

1

读入与存储

n 张地毯各存 4 个数 (a 、 b 、 g 、 k 四个数组)，询问点 x 、 y 单独读。注意 n 可以为 0—— ans 初值必须就是 -1 。

2

顺序扫描，命中就覆盖

从 1 到 n 逐张判断： $a_i \leq x \leq a_i + g_i$ 且 $b_i \leq y \leq b_i + k_i \rightarrow ans = i$ 。后铺的直接覆盖 ans ，"最上面"自动成立。

3

或者倒着扫，找到即停

从 n 往 1 扫，第一张覆盖的就是答案——两种写法等价，随你喜欢。复杂度都是 $O(n)$ 。

一句话

题目只给了"后盖前"一条规则——识别出它等价于"取编号最大的覆盖者"，这题就已经做完一半。

方法拆解 + 样例手推：3 与 -1

两个考点

- ① 边界含等号： $a \leq x \leq a+g$ 、 $b \leq y \leq b+k$ ，四个 $\leq$ 都不能少——踩在地毯边上也算“被覆盖”，漏一个等号就 WA。
- ② 别开二维格子：坐标可达 10^5 ，画 $10^5 \times 10^5$ 的网格 $\approx 10^{10}$ 格，内存直接爆。逐张判断矩形是否含点即可——模拟 $\neq$ 把世界画出来。

状态一览

唯一的“状态”就是答案 ans（初值 -1）——比前两题还少，但边界思维一样不能少。

地毯 i	x 范围	y 范围	(2,2) 在内?	ans
1 (1,0,2,3)	1..3	0..3	√ 命中	1
2 (0,2,3,3)	0..3	2..5	√ 命中	2
3 (2,1,3,3)	2..5	1..4	√ 命中	3

第二次询问 (4,5) → -1

地毯 1: $4 > 3$ ✕; 地毯 2: $4 > 3$ ✕; 地毯 3: $4 \leq 5$ √ 但 $5 > 4$ ✕。三张全 miss → 输出 -1 √（脚本验证一致）。

易错

x 方向是 $a+g$ （不是 $a+g-1$ ），y 方向是 $b+k$ ；两个条件必须同时成立； $n=0$ 时输出 -1，别让循环炸掉。

参考代码：十五行，全在边界上

第 2 课 · 多规则、多状态模拟

T05 · 铺地毯 (P1003)

```
#include <iostream>
using namespace std;
int a[10005], b[10005], g[10005], k[10005];
int main() {
    int n; cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i] >> b[i] >> g[i] >> k[i];
    int x, y; cin >> x >> y;
    int ans = -1; // 初值就是 -1 (含 n=0)
    for (int i = 1; i <= n; i++)
        if (a[i] <= x && x <= a[i] + g[i] &&
            b[i] <= y && y <= b[i] + k[i]) // 含边界: 四个 <=
            ans = i; // 后铺的盖前面
    cout << ans << endl;
    return 0;
}
```

覆盖即答案

ans = i 一行完成"后盖前": 编号大的总是最后写入。想用倒序版本, 就把循环反过来、命中即 break。

判断条件的读法

两行 if 就是"点在矩形内"的展开式: x 被夹在 a 与 a+g 之间、y 被夹在 b 与 b+k 之间——逐个对照, 不要凭感觉写。

数组的容量意识

$n \leq 10^4$, 数组开 10005 刚好; 若题目说 $n \leq 10^5$ 就要同步放大——数组大小跟着数据范围走。

提交提示

课后提交洛谷 P1003。两组官方样例 (3 / -1) 与全部测试点均通过对拍脚本验证。

T06 | 玩具谜题 (P1563)

第 2 课 · 多规则、多状态模拟

NOIP2016 普及组 · 难度 ★★★

题面描述

n 个小玩具围成一圈，每个玩具有**朝向**（0 朝内、1 朝外）和**职业名**。医生从第 1 个玩具出发，依次执行 m 条指令：每条为 $s\ x$ —— $s=0$ 表示"左数"、 $s=1$ 表示"右数"，数 x 个玩具。

方向判定：玩具朝内 + 左数（或朝外 + 右数）→ 沿编号减小方向数；玩具朝内 + 右数（或朝外 + 左数）→ 沿编号增大方向数。围成圈，首尾相接。

求 m 条指令执行完，最后停在第几个玩具、输出它的职业名。

思路预告

两个"方向"缠在一起：玩具的朝向 $\times$ 指令的方向——画一张 2×2 表理清，加还是减一目了然。

输入格式

第一行 $n\ m$ ；接下来 n 行"朝向 职业名"；接下来 m 行"指令 s 数值 x "。

输出格式

一行一个字符串：最终玩具的职业名。

数据范围

$n, m \leq 10^5$ ； x 可达 10^9 级——跨多圈，取模是刚需。

官方样例 1 → writer

```
7 3
0 singer  0 reader  0 mengbier
1 thinker  1 archer  0 writer  1 magician
0 3  1 1  0 2
（官方输入为每行一条；此处为排版紧凑展示）→ 输出 writer
```

思路梳理：2×2 表理出"同减异加"

四种组合，两种走向

玩具朝向 \ 指令	左数 (s=0)	右数 (s=1)
朝内 (0)	编号减小 (同 → 减)	编号增大 (异 → 加)
朝外 (1)	编号增大 (异 → 加)	编号减小 (同 → 减)

三步主循环

- ① 从 0 号玩具 (第 1 个) 出发, cur 记当前位置。
- ② 每条指令: **朝向 == 指令 → 减, 朝向 != 指令 → 加**——"同减异加"四个字。
- ③ m 条做完, 输出 job[cur]。

为什么必须取模?

x 可达 10^9 : 绕圈几十万次也是回到原地——先 $x \% n$, 一步跨到位。

减法可能变负: $cur - x \% n$ 最少到 $-(n-1)$, 所以写 $((cur - x \% n) \% n + n) \% n$, 负数也安全回卷。

一句话

这题没有复杂的数据结构, 考的就是把 2×2 的方向关系压缩成一行 if, 再用取模处理"绕圈"和"变负"两个坑。

方法拆解 + 样例手推：singer → archer → thinker → writer

玩具圈（样例 1，编号 0 起）

0 singer（朝内）

1 reader（朝内）

2 mengbier（朝内）

3 thinker（朝外）

4 archer（朝外）

5 writer（朝内）

6 magician（朝外）

手推三步走

① 当前玩具的朝向 ② 与指令同异 ③ 加还是减——一行一行落笔。

步	指令	当前玩具（朝向）	同/异	落点
起点	—	singer（内）	—	0
①	0 3（左数 3）	singer（内）	同 → 减 3	0-3 ≡ 4 archer
②	1 1（右数 1）	archer（外）	同 → 减 1	4-1 = 3 thinker
③	0 2（左数 2）	thinker（外）	异 → 加 2	3+2 = 5 writer

三个算式看仔细（脚本验证一致）

① $(0 - 3 + 7) \% 7 = 4$ ：负数靠 +n 回卷 → archer（朝外）；② archer 朝外、指令右数 → 同 → 减 → thinker；③ thinker 朝外、指令左数 → 异 → 加 → **writer** ✓。样例 2 同法得 y。

参考代码："同减异加"一行 if

第 2 课 · 多规则、多状态模拟

T06 · 玩具谜题 (P1563)

```
#include <iostream>
#include <string>
using namespace std;
int dirn[100005];
string job[100005];
int main() {
    int n, m; cin >> n >> m;
    for (int i = 0; i < n; i++) cin >> dirn[i] >> job[i];
    int cur = 0; // 从第 1 个玩具开始
    for (int i = 0; i < m; i++) {
        int s, x; cin >> s >> x;
        if (dirn[cur] == s) // 同减: 逆着编号走
            cur = ((cur - x % n) % n + n) % n;
        else // 异加: 顺着编号走
            cur = (cur + x) % n;
    }
    cout << job[cur] << endl;
    return 0;
}
```

字符串直接进数组

string job[100005] 一行搞定职业名—— $n \leq 10^5$ 个短字符串完全放得下。

回卷公式的来历

$((cur - x \% n) \% n + n) \% n$: 内层取模防溢出、中间可能为负、+n 拉回非负、外层再 %n——四步缺一不可。

别逐个数

如果真的一步一步数 x 次, 10^9 步直接 TLE——取模一步到位, 这就是"看清数据范围再写循环"。

提交提示

课后提交洛谷 P1563。两组官方样例 (writer / y) 与全部测试点均通过对拍脚本验证。

T07 | 两只塔姆沃斯牛 (P1518)

第 2 课 · 多规则、多状态模拟

USACO 经典 · 难度 ★★★

题面描述

10×10 的农场网格：**F** 农夫、**C** 牛、***** 障碍、**.** 空地。两人**同时**行动：每分钟各朝当前方向走一格；若前方出界或撞障碍，则**原地顺时针转向**（上→右→下→左），这一分钟不前进。两者初始都朝上。

每分钟结束后检查：若两人位于同一格 → 输出经过的分钟数；若永远不可能相遇 → 输出 0。

思路预告

两套移动规则同时跑不稀奇——稀奇的是"永不相遇"怎么判断：靠**状态**见过没有。

输入 / 输出

输入 10 行、每行 10 个字符；输出一个整数：相遇分钟数或 0。

为什么一定能在有限步内判断？

两人各由"位置×方向"决定：每个 $(10 \times 10 \times 4) = 400$ 种，组合 $400 \times 400 = 160000$ 种状态。状态一重复，之后全是重播 → 输出 0。

官方样例 → 49 (脚本已验证)

```
*..*....
.....*..
...*...*..
.....
...*.F....
*....*...
...*
..C.....*
...*...
...*...
...*
```

思路梳理：六元组状态，"见过"即死循环

第 2 课 · 多规则、多状态模拟

T07 · 两只塔姆沃斯牛 (P1518)

1

一方的移动规则写成 step 函数

前方 $(x+dx[d], y+dy[d])$ 出界或是 * $\rightarrow$ 原地转 $(d+1)\%4$; 否则前进一格。方向编码 上0 右1 下2 左3, $dx=0$ 、 $dy=-1$ 。农夫和牛各调用一次。

2

每分钟的固定顺序

先判相遇 (同格 $\rightarrow$ 输出 t) ; 再查状态 (fx,fy,fd,cx,cy,cd) 是否见过 (见过 $\rightarrow$ 输出 0) ; 然后记录状态、双方各走一步。

3

循环上界 160000

状态总数只有 160000 种——只要循环超过这个数还没结束，一定已经重复过。把上界写进 for，比 while(1) 更安心。

一句话

"永远不"没法直接判断，但状态空间有限——见过的状态再次出现，就是死循环的铁证。

方法拆解 + 手推前 5 分钟

状态怎么存

六元组: (fx, fy, fd, cx, cy, cd)——两人位置加方向，一步都不能少。

vis 六维数组: bool vis[10][10][4][10][10][4], 共 160000 格、约 160KB, 开得下。

起点: 样例中 F 在 (4,5)、C 在 (7,2), 都朝上 (d=0)。

转向口诀

出界或撞障碍 → 原地转 90° (d+1)%4, 这一分钟不走。别绕过障碍"滑行"。

分钟 t	农夫 (行,列 · 朝向)	牛 (行,列 · 朝向)	说明
0	(4,5) · 上	(7,2) · 上	初始; 不同格, 记入 vis
1	(3,5) · 上	(6,2) · 上	双双北进
2	(2,5) · 上	(5,2) · 上	继续北进
3	(1,5) · 上	(4,2) · 上	农夫进入第 1 行
4	(0,5) · 上	(3,2) · 上	农夫抵达第 0 行
5	(0,5) · 右	(2,2) · 上	农夫北上出界 → 原地转向

第 5 分钟是重点

农夫在 (0,5) 朝上, 前方出界 → 只转向不前进, 方向变"右"; 牛不受影响继续北上。之后 44 分钟两人继续按规则走, 最终在第 49 分钟同格相遇 √ (脚本验证一致)。

参考代码：六维数组判重

第 2 课 · 多规则、多状态模拟

T07 · 两只塔姆沃斯牛 (P1518)

```
#include <iostream>
using namespace std;
char g[10][11];
bool vis[10][10][4][10][10][4]; // 六维: 状态见过 = 死循环
int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1}; // 上右下左
void step(int &x, int &y, int &d) {
    int nx = x + dx[d], ny = y + dy[d];
    if (nx < 0 || nx > 9 || ny < 0 || ny > 9 || g[nx][ny] == '*')
        d = (d + 1) % 4; // 出界或撞障碍: 原地转向
    else { x = nx; y = ny; }
}
int main() {
    int fx, fy, cx, cy, fd = 0, cd = 0;
    for (int i = 0; i < 10; i++) cin >> g[i];
    for (int i = 0; i < 10; i++)
        for (int j = 0; j < 10; j++) {
            if (g[i][j] == 'F') { fx = i; fy = j; }
            if (g[i][j] == 'C') { cx = i; cy = j; }
        }
    for (int t = 0; t <= 160000; t++) {
        if (fx == cx && fy == cy) { cout << t; return 0; } // 相遇
        if (vis[fx][fy][fd][cx][cy][cd]) { cout << 0; return 0; }
        vis[fx][fy][fd][cx][cy][cd] = true;
        step(fx, fy, fd); step(cx, cy, cd); // 双方同时各走一步
    }
    cout << 0; return 0;
}
```

引用传参

step(int &x, int &y, int &d) 里的 & 让函数直接改外面的变量——农夫、牛共用一个函数，各改各的。

六维数组开得下

$10 \times 10 \times 4 \times 10 \times 10 \times 4 = 160000$ 个 bool $\approx$ 160KB——用空间换“永不相遇”的判断力。

判重的次序

先查相遇、再查 vis、然后记 vis、最后才走——顺序错一格，样例都过不了。

提交提示

课后提交洛谷 P1518。官方样例（49）与全部测试点均通过对拍脚本验证。

T08 | 神奇的幻方 (P2615)

题面描述：按规则填 $1..n^2$

$n \times n$ 矩阵 (n 为奇数)。1 填在第一行正中间；之后每个数 K 填在 $K-1$ 的右上方格 (行减一、列加一)，并按下面四条规则处理越界与冲突：

- ① $K-1$ 在第一行但不在最后一列 $\rightarrow K$ 填在最后一行、 $K-1$ 右一列；
- ② $K-1$ 在最后一列但不在第一行 $\rightarrow K$ 填在第一列、 $K-1$ 上一行；
- ③ $K-1$ 在第一行最后一列 (右上角) $\rightarrow K$ 填在 $K-1$ 正下方；
- ④ 其余情况：右上方格未填 $\rightarrow$ 填右上方格；已填 $\rightarrow$ 填 $K-1$ 正下方。

四条规则互斥且覆盖所有情形——每一步恰好命中一条。

思路预告

多规则题的教科书：规则多但互斥——用 if / else if 链逐条翻译，一条不漏。

输入 / 输出

输入一个奇数 n ；输出 n 行 n 列的幻方，同一行数字用空格隔开。

数据范围

$1 \leq n \leq 39$ 且 n 为奇数；总填数 $\leq 39^2 = 1521$ ， $O(n^2)$ 轻松通过。

官方样例 $n=3 \rightarrow$ 三阶幻方

输出

8 1 6
3 5 7
4 9 2

每行、每列、两条对角线之和都等于 15。对拍脚本对 $n=25$ 也验证了幻方性质 (行/列/对角和 7825，恰为 $1..625$)。

思路梳理：四条规则 = 一条 if 链

第 2 课 · 多规则、多状态模拟

T08 · 神奇的幻方 (P2615)

主循环骨架

- ① **起点**：坐标从 1 开始，1 填在 $(1, (n+1)/2)$ 。
- ② **逐个填 $K = 2..n^2$** ：看 $K-1$ 所在的 (x, y) 命中哪条规则，算出 K 的位置——填一个、走一步。
- ③ **判空**：数组初值全 0，"右上方格未填"就是 $a[x-1][y+1] == 0$ ——"填过没有"直接问数组。

翻译对照 (伪代码 $\rightarrow$ C++)

- ③ 右上角 ($x==1$ 且 $y==n$) $\rightarrow$ 正下: $x++$
- ① 第一行 ($x==1$) $\rightarrow$ 最后一行、右一列: $x=n, y++$
- ② 最后一列 ($y==n$) $\rightarrow$ 上一行、第一列: $x--, y=1$
- ④ 右上为空 $\rightarrow$ 右上: $x--, y++$; 已填 $\rightarrow x++$

注：③ 与 ①② 的条件互斥，判定先后不影响正确性，但四条都要写。

易错

漏写 ③ (右上角) 是最常见的丢分点：只写 ①② 的话， $(1, n)$ 会同时"看起来"命中 ① 或 ② 的变形，越界访问 $a[0]$ 或 $a[n+1]$ 。

一个惊喜

全程不用取模！四条规则的边界条件恰好把"第一行、最后一行、第一列、最后一列"四种边缘全接住了。

方法拆解 + 手推 n=3：九步填出幻方

K	K 填在哪 (行,列)	命中规则 (决定下一格)	下一格
1	(1,2) 第一行正中间 (起点)	第一行 → ①	(3,3)
2	(3,3)	最后一列 → ②	(2,1)
3	(2,1)	④ 右上 (1,2) 已被 1 占 → 正下	(3,1)
4	(3,1)	④ 右上 (2,2) 空 → 右上	(2,2)
5	(2,2)	④ 右上 (1,3) 空 → 右上	(1,3)
6	(1,3)	右上角 → ③ 正下	(2,3)
7	(2,3)	最后一列 → ②	(1,1)
8	(1,1)	第一行 → ①	(3,2)
9	(3,2)	$n^2=9$, 填完收工	—

● 九步走完，得到 **8 1 6 / 3 5 7 / 4 9 2** ✓（脚本验证一致）。手推时把"命中规则"一列写出来，代码就是照着这列抄。

跟着表走一遍

K=3 是第一次"④ 撞车"：右上的 (1,2) 已有 1 → 只能正下。K=6 撞上 ③ 右上角特判——这也是初学者最容易漏的一步。

复盘

九步里 ① 用了 2 次、② 用了 2 次、③ 用了 1 次、④ 用了 4 次（含 2 次判空失败走正下）——四条规则一个都不能少。

参考代码：if - else if 链的教科书

第 2 课 · 多规则、多状态模拟

T08 · 神奇的幻方 (P2615)

```
#include <iostream>
using namespace std;
int a[45][45];
int main() {
    int n; cin >> n;
    int x = 1, y = (n + 1) / 2;           // 1 放第一行正中间
    for (int k = 1; k <= n * n; k++) {
        a[x][y] = k;
        if (x == 1 && y == n) x++;        // ③ 右上角 → 正下
        else if (x == 1) x = n, y++;      // ① 第一行 → 最后一行
        else if (y == n) x--, y = 1;       // ② 最后一列 → 第一列
        else if (a[x-1][y+1] == 0) x--, y++; // ④ 右上为空 → 右上
        else x++;                          // ④ 否则 → 正下
    }
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            cout << a[i][j] << (j == n ? "\n" : " ");
    return 0;
}
```

逐条对应第 38 页

五行 if - else if 与四条规则一一对应——题目规则有几条，代码分支就有几个，多一条少一条都错。

判空是一切的前提

全局数组初值全 0， $a[x-1][y+1]==0$ 即“没填过”——不需要额外的 bool 数组。

输出的小心思

行末不能有多余空格：j==n 时换行、否则空格——三目运算符一行搞定。

提交提示

课后提交洛谷 P2615。n=3 样例与 n=25 幻方性质（行/列/对角和 7825）均已脚本验证。

T09 | 直播获奖 (P7072)

第 2 课 · 多规则、多状态模拟

CSP-J 2020 T2 · 难度 ★★★

题面描述

NOIP 获奖率规定为 $w\%$ 。选手**按报名顺序逐一**提交成绩 a_i ($0..600$ 的整数)。每当第 i 位选手提交后，实时输出**当前获奖分数线**。

分数线规则：当前已提交 x 人时，获奖名额 $p = \lfloor w \times x / 100 \rfloor$ ，且 p 至少为 1；分数线 = **成绩从高到低第 p 名选手的成绩**（同分并列时，第 p 名取其中最低的分数）。

共输出 n 行。

思路预告

每人提交都重新排名？ $n \leq 10^5$ ，排序排不起——值域只有 $0..600$ ，用**桶计数**：把"排序"变成"数人头"。

输入格式

第一行 n, w ；第二行 n 个整数 $a_1 \dots a_n$ （按提交顺序）。

输出格式

n 行，第 i 行为第 i 位选手提交后的分数线。

数据范围

$1 \leq n \leq 10^5$ ； $0 \leq a_i \leq 600$ 。

官方样例（脚本已验证）

样例 1 ($w=60$) → 输出
10 60
200 300 400 500 600 600 0
300 200 100
→ 200 300 400 400 400
500 400 400 300 300

样例 2 ($w=30$) → 输出
10 30
100 100 600 100 ... 100
→ 100 100 600 600 600
600 100 100 100 100

思路梳理：桶计数，从高往低数人头

第 2 课 · 多规则、多状态模拟

T09 · 直播获奖 (P7072)

1

桶：cnt[v] = 成绩 v 的人数

成绩只有 0..600 一共 601 种取值——直接当数组下标。每提交一人， $\text{cnt}[a_i]++$ ， $O(1)$ 完成"插入"。

2

名额 p：整数除法 + 下限保护

$p = w \times i / 100$ （整数除法自动向下取整）；若 $p < 1$ 则 $p = 1$ ——第一名永远获奖。

3

找分数线：从 600 往下累加

seen 从 0 起、v 从 600 到 0 逐档累加 $\text{cnt}[v]$ ，第一次 $\text{seen} \geq p$ 的 v 就是分数线——"第 p 名"=数到第 p 个人时所在的档。

一句话

值域小 = 桶的机会。600 档 $\times n$ 次询问，最坏 6×10^7 次简单加法，稳过——排序则要 $n \cdot n \cdot \log n$ ，直接爆炸。

方法拆解 + 手推样例 1 (w=60)

i	新成绩	$p = \max(1, 60i/100)$	从高到低第 p 名	分数线
1	200	$0 \rightarrow 1$	200	200
2	300	1	300	300
3	400	1 (1.8 取整)	400	400
4	500	2 (2.4 取整)	500,400 $\rightarrow$ 第 2 个	400
5	600	3	600,500,400 $\rightarrow$ 第 3 个	400
6	600	3 (3.6 取整)	600,600,500 $\rightarrow$ 第 3 个	500
7	0	4 (4.2 取整)	$\cdots$ 500,400,0 $\rightarrow$ 第 4 个	400
8	300	4 (4.8 取整)	600,600,500,400	400
9	200	5 (5.4 取整)	$\cdots$ 400,400,300 $\rightarrow$ 第 5 个	300
10	100	6	$\cdots$ 300,200,100 $\rightarrow$ 第 6 个	300

输出逐行对照官方答案

200 300 400 400 400 500 400 400 300 300 $\checkmark$
(脚本验证一致)。样例 2 同法得 100 100 600
600 600 600 100 100 100 100。

两个易错

- ① p 是"当前人数 $\times$ w"再取整，不是总人数固定值；
- ② 同分并列时第 p 名取较低分——累加 cnt 再判 $seen \geq p$ 恰好实现了这一点。

参考代码：桶计数三板斧

第 2 课 · 多规则、多状态模拟

T09 · 直播获奖 (P7072)

```
#include <iostream>
using namespace std;
int cnt[605];
int main() {
    int n, w; cin >> n >> w;
    for (int i = 1; i <= n; i++) {
        int a; cin >> a;
        cnt[a]++;                // 入桶: O(1)
        int p = w * i / 100;      // 整数除法自动向下取整
        if (p < 1) p = 1;         // 至少 1 人获奖
        int seen = 0, ans = 0;
        for (int v = 600; v >= 0; v--) { // 从高分往低分数人头
            seen += cnt[v];
            if (seen >= p) { ans = v; break; }
        }
        cout << ans << "\n";      // 每人提交后输出一行
    }
    return 0;
}
```

桶的思想一以贯之

第 9 页 T01 用队列管"时间窗"，这里用桶管"值域"——都是用数据结构替人记账。

p 的三个细节

$w \times i$ 最大 $100 \times 10^5 = 10^7$ ，int 装得下；整数除法即向下取整； $p < 1$ 时拉回 1。

内层循环最多 601 步

找到就 break——平均更快；就算跑满也只有 $601 \times 10^5 \approx 6 \times 10^7$ 次加法。

提交提示

课后提交洛谷 P7072。两组官方样例与全部测试点均通过对拍脚本验证。

T10 | 小熊的果篮 (P7912)

题面描述

n 个苹果排成一排，第 i 个苹果的种类为 a_i (0 或 1)。每一轮，小熊把当前苹果序列中**每一个**"连续同类型段"的最左边一个苹果**同时**拿走。

连续段：相邻且种类相同的极大连续块。例如 1 1 0 0 1 1 1 0 1 1 0 0 分成 6 段，段首是 1、3、5、8、9、11。

输出每轮拿走的苹果编号（按从小到大），每轮一行，直到苹果拿完。

思路预告

段会变短、段会合并，段首也跟着变——这道 T4 考的是把"同时拿走"翻译成"先拍快照、轮末统一摘链"。

输入 / 输出

输入：第一行 n ，第二行 n 个 0/1。输出：若干行，每轮一行编号（空格隔开）。

数据范围

n 最大约 2×10^5 ； $a_i \in \{0, 1\}$ 。逐轮重扫全数组是 $O(n^2)$ ，会超时。

官方样例（脚本已验证）

样例 1 ($n=12$) → 输出
1 1 0 0 1 1 1 0 1 1 0 0
→ 1 3 5 8 9 11
2 4 6 12
7
10

样例 2 ($n=20$) → 4 轮
1 5 8 11 13 14 15 17
2 6 9 12 16 18
3 7 10 19
4 20

思路梳理："同时"二字的工程化

第 2 课 · 多规则、多状态模拟

T10 · 小熊的果篮 (P7912)

1

每轮拿的是"段首"，不是"第奇数个"

对当前剩下的序列重新分段，每段最左边一个被拿走。段会缩短，相邻同段还会合并——状态每轮都在变。

2

关键：先拍快照，轮末再摘链

本轮只处理"轮初队列里"的那些段头（快照 k 个）；拿走的苹果**先标记、本轮不摘链**，等本轮全部判断完，再统一从链上摘掉——这就是"同时"。

3

三件套：链表 + 标记 + 段头队列

pre / nxt 双向链表把"摘掉一个苹果"降到 $O(1)$ ； $gone[]$ 标记已拿走；队列里只放"可能成为段头"的编号，被拿苹果的右邻是下一个候选。

一句话

拿走一个苹果只影响它的左右邻居——链表是为"邻居关系经常变"量身定做的状态结构。

方法拆解 + 四轮手推（样例 1）

轮	拿走的编号	剩下的序列
1	1 3 5 8 9 11	2 4 6 7 10 12
2	2 4 6 12	7 10
3	7	10
4	10	(空)

段首怎么"接棒"?

拿走段首 h 时看它的右邻 r : 若 $a[r] == a[h]$, r 就是这段的新段首, 入队等待下一轮——轮 1 拿走 5 后, 6 接棒入队。

复盘

每个苹果最多"入队两次" (初始段头 + 一次接棒), 总工作量 $O(n)$ ——这就是 T4 的满分复杂度。两个样例均为脚本验证的四轮输出。

轮 2 为什么没有 10?

轮 2 剩下 2 4 6 7 10 12, 重新分段: [2] [4] [6,7,10] [12]——10 在段 [6,7,10] 的中间, 不是段首, 所以轮 2 只拿段首 2、4、6、12。这行判断在代码里就是: $pre[h]$ 的种类还和我一样 $\rightarrow$ 跳过。

参考代码：链表 + 快照队列

第 2 课 · 多规则、多状态模拟

T10 · 小熊的果篮 (P7912)

```
#include <iostream>
using namespace std;
const int N = 200005; int a[N], pre[N], nxt[N], basket[N];
bool gone[N];
int q[400005], head = 0, tail = 0; // 每个苹果最多入队两次
int main() {
    int n; cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> a[i]; pre[i] = i - 1; nxt[i] = i + 1;
        if (i == 1 || a[i] != a[i-1]) q[tail++] = i; // 初始段头
    }
    while (head < tail) {
        int k = tail - head, cnt = 0; // 快照: 本轮只处理 k 个
        for (int t = 0; t < k; t++) {
            int h = q[head++];
            if (gone[h] || (pre[h] >= 1 && a[pre[h]] == a[h])) continue;
            basket[cnt++] = h; int r = nxt[h];
            if (r <= n && a[r] == a[h]) q[tail++] = r; // 右邻接棒
        }
        for (int t = 0; t < cnt; t++) { // 轮末统一摘链 = "同时"
            int h = basket[t];
            nxt[pre[h]] = nxt[h]; pre[nxt[h]] = pre[h]; gone[h] = true;
        }
        for (int t = 0; t < cnt; t++) cout << basket[t] << (t == cnt - 1 ? "\n" : " ");
    }
    return 0;
}
```

快照 = 循环变量 k

k 在轮初定死，循环里 tail 还在涨也不管——新接棒的段头自动留给下一轮。

两行链表摘除

$\text{nxt}[\text{pre}[\text{h}]] = \text{nxt}[\text{h}]$ 、 $\text{pre}[\text{nxt}[\text{h}]] = \text{pre}[\text{h}]$ ——双向链表 $O(1)$ 删除；比 vector 的 erase 快得多。

队列要开 $2n$

初始段头最多 n 个，接棒再入队最多 n 个——q 开 400005，数组越界就 RE。

提交提示

课后提交洛谷 P7912。两组官方样例（四轮输出）均通过对拍脚本验证。

常见错误门诊：四个高频症状

WA 答案错误 —— 规则翻错、优先级乱

典型：P5661 把 $t - t_0 \leq 45$ 写成 $<$ ；P2615 规则①③判断顺序颠倒；P1328 胜负表抄错一格。

对策：拿官方样例手推一遍，对照规则清单逐条核对每个分支的条件与先后。

TLE 运行超时 —— 全量重扫、复杂度爆

典型：P5661 每买一张票把票夹整体搬移成 $O(n^2)$ ；P7072 每来一个人就把全桶重扫一遍。

对策：先估算 n 乘循环层数，超过 10^8 就换队列、桶或链表，只动"该动"的那部分。

RE 运行错误 —— 越界、除零

典型：P1518 判重数组少开一维；P7912 队列只开 $2n$ 不够；数组比数据范围小一号。

对策：数组大小按数据范围上限再 +5；下标先判范围再使用，模运算注意负数。

格式 格式错误 —— 空行空格、边界差一

典型：P1042 末局要照原样输出、局间空行漏掉；P1003 覆盖判断少写一个等号。

对策：逐字符对照样例输出；凡是 $\leq$ 还是 $<$ 的边界，单独造一组"恰好相等"的数据。

课后任务与第 3 课预告

第 2 课 · 多规则、多状态模拟

五档任务 · 洛谷提交 · 截止周五 22:00

必做 课上四题提交 AC

T02 接水 P1190 · T03 乒乓球 P1042 · T04 生活大爆炸 P1328 · T05 铺地毯 P1003——课上未完成的当堂补完并提交。

复盘 脱稿重写 T01 公交换乘

合上课件，独立复述"清队头 → 找票 → 花钱"三步与过期条件 $t > t_0 + 45$ ，再默写 26 行参考代码。

争取 三道真题补全

T06 智者的游戏 P1563 · T07 两只塔姆沃斯牛 P1518 · T08 神奇的幻方 P2615——每题先手推样例，再落笔写代码。

挑战 两道压轴选做

T09 直播获奖 P7072 · T10 小熊的果篮 P7912——体会桶计数与"链表 + 段头队列"是怎样把时间省下来的。

预习 第 3 课热身思考

从 n 个数里选 k 个，使和为质数，共有多少种选法？先想想"枚举什么、按什么顺序"。（P1036 选数）

NEXT LESSON · LESSON 03

枚举算法入门

枚举三问，下节课开讲：

- ① 枚举什么状态？
- ② 按什么顺序枚举？
- ③ 怎样提前排除不可能（剪枝）？

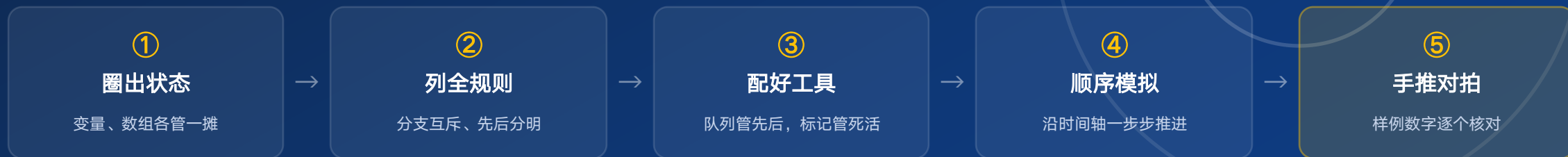
热身真题：P1036 选数（NOIP 2002 普及组）

提交方式与截止

全部在洛谷提交，通过官方样例与测试点才算完成；**截止周五 22:00**。卡壳时先回看课上手推表，再向老师提问。

找全状态，分好规则

多规则、多状态模拟 · 本课回顾：十道洛谷真题，一套通用流程



模拟题不怕长，怕的是状态漏管、规则漏翻——把这两件事做扎实，你就读懂了出题人的剧本。

授课班级

中预 2026 级 B 班

课后截止

周五 22:00 · 洛谷提交

第 3 课《枚举算法入门》· 下期见

02